

NAPCO Nucleus AI Agent

Operational Blueprint

Architect

Mohammad Kamrul Hasan

Focus

Project Management | Test Automation | DevOps / Release Automation

April 2026

Overview

What NAPCO Nucleus AI Agent does and why it exists.

NAPCO Nucleus AI Agent is a Claude-native AI deputy built to remove the daily friction from three operational dimensions inside NAPCO: **Project Management** (requirement intake and stakeholder reporting), **Test Automation** (API functional, integration, load, and end-to-end testing of the MVP Access platform), and **DevOps / Release Automation** (the nightly TFS pull, .NET build, and IIS deploy that primes the test cycle). It runs on a self-hosted Windows VM under GitHub Actions, calls Claude Opus 4.7 through the locally authenticated Claude CLI, and ships its outputs as PDF email reports, GitLab issues, and an optional Teams digest.

The architecture is borrowed from the Digital Deputy framework but tuned for a single-engineer use case at NAPCO. The agent does the heavy lifting (transcription, dedupe, splitting, analysis, reporting) so that the engineer stays focused on judgment work: which regression is real, which requirement needs clarification, which release is ready to ship.

All seven workflows are live in production, scheduled on the self-hosted runner, and observable through a single SQLite memory store and one consolidated morning email.

Core Principles

The rules every workflow inherits.

Claude does the reasoning, Python does the doing.

Every agent run is a single Claude Agent SDK turn. The Python tools handle deterministic work (file reads, API calls, SQL writes, SMTP sends); Claude handles judgment (splitting requirements, analyzing failures, drafting reports).

Memory at every gate.

Every prompt opens with a mandatory check-in to *nucleus_memory.db*. The agent never cold-starts: it recalls past activity, prior requirements, and recent test trends before doing fresh work.

Idempotent by construction.

Requirements are deduped via FTS5 fuzzy match before they hit GitLab. Test runs are keyed by date and suite. Re-running a workflow is safe; it will not double-write.

One email per day.

The four test workflows write results to memory but do *not* send per-run emails. The Daily Report consolidates everything into a single morning PDF so the team's inbox stays clean.

No API keys in CI.

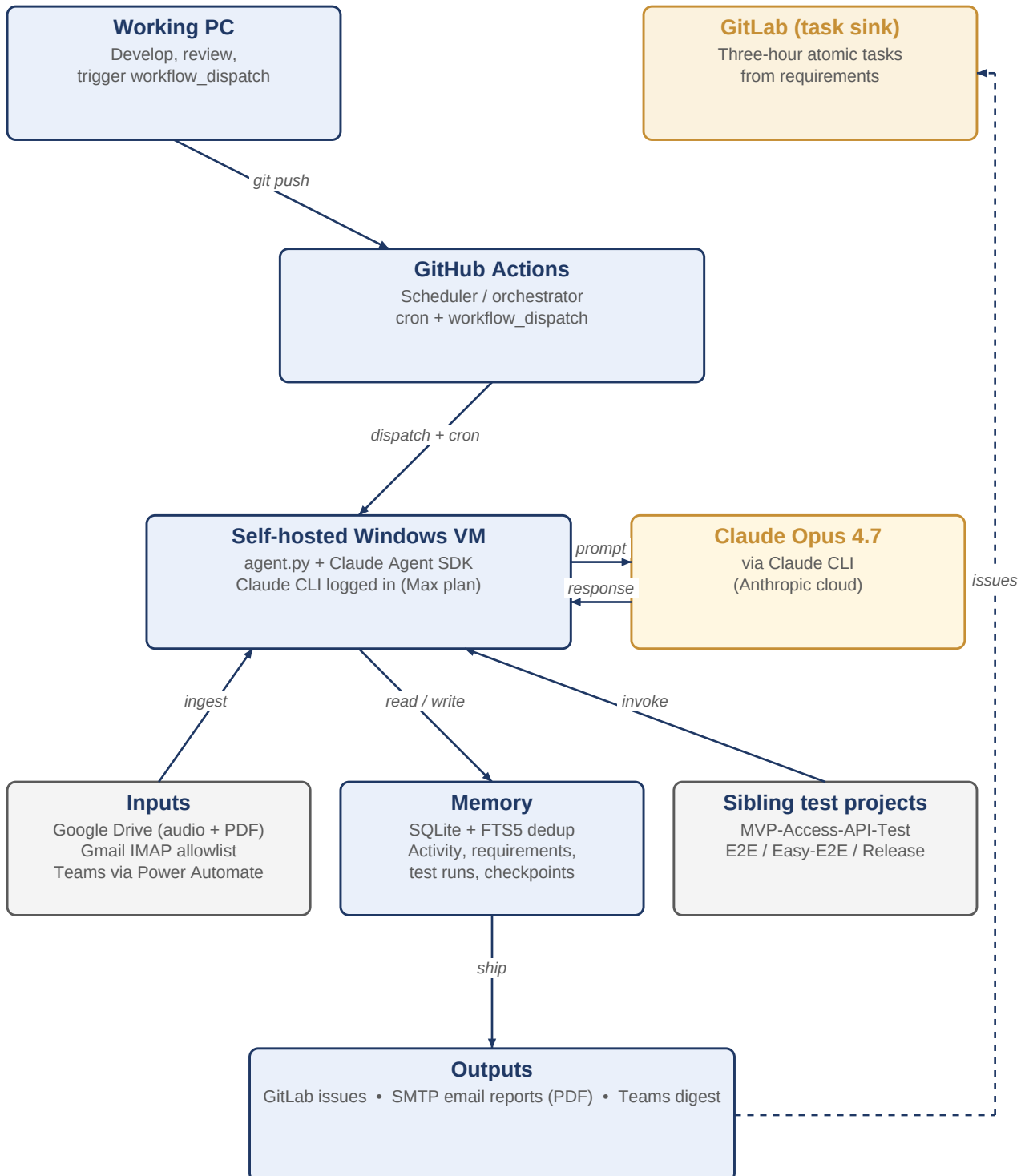
Reasoning is billed against the engineer's Claude Max subscription via the locally authenticated CLI on the self-hosted runner. The cloud runner has no Anthropic credentials and never will.

Dry run is a first-class mode.

--dry-run on any task runs every step except the mutating ones (no SMTP send, no GitLab create, no git push). Used on every prompt change before promoting it to the schedule.

System Architecture

How the pieces talk to each other when a workflow fires.



The working PC is the development surface. GitHub Actions is the scheduler that fires workflows on cron or on demand. Each workflow targets the self-hosted Windows VM, where *agent.py* loads the shared *system.md* plus the task-specific prompt and runs one Claude Agent SDK turn against Claude Opus 4.7 (via the locally authenticated Claude CLI). Claude calls back into 36 MCP tools (memory, requirements, tests, report, files) which read inputs from Google Drive, Gmail IMAP, and four sibling test projects, write state to *nucleus_memory.db*, and ship outputs as GitLab issues, SMTP-delivered PDF reports, and an optional Teams digest.

Architecture at a Glance

Six layers, one consistent operational posture.

Layer	Description
Reasoning core	Claude Agent SDK running against Claude Opus 4.7. The agent handles all judgment work: requirement splitting, fuzzy dedup gating, test-failure root cause, report drafting.
Persistent memory	SQLite (nucleus_memory.db) with FTS5 indices over activity_logs, requirements_seen, test_run_history, email_checkpoints, drive_processed. Committed to the repo so memory travels with git clone.
Tools layer	36 MCP tools split across five submodules — memory, requirements, tests, report, files. The agent calls them; they wrap deterministic helpers (IMAP, Drive, Whisper, GitLab, SMTP, Newman, Locust, pytest, Playwright).
Orchestration	GitHub Actions on a self-hosted Windows runner. Seven workflow files: six task workflows plus a probe-runner health check. Concurrency groups prevent overlapping runs of the same task.
Inputs	Gmail IMAP with sender allowlist, Google Drive folder for audio meetings (routed through Groq Whisper) and PDFs (parsed via pypdf), and four sibling test projects under E:/Projects/ that the test workflows invoke.
Outputs	GitLab issues (three-hour atomic tasks), PDF email reports through SMTP, and an optional Microsoft Teams digest webhook for at-a-glance status.

Dimension 1 — Project Management

Requirement flow and stakeholder visibility.

Workflow 1.1 — Requirement Management

Process	Fires every two hours during business hours (09:00–17:00 BDT, Sunday–Thursday). Polls Gmail IMAP for emails from an allowlisted sender set and scans a designated Google Drive folder for new meeting audio (routed through Groq Whisper, model whisper-large-v3-turbo, 25 MB per file) and PDFs (parsed via pypdf).
Action	Claude reads the requirement inbox, fuzzy-matches each candidate against the FTS5 index of past requirements to suppress duplicates, splits approved items into ~3-hour atomic tasks with acceptance criteria, and calls the GitLab v4 REST API.
Result	Tasks land in the GitLab backlog with title-based idempotent dedup. Drive files and email message-ids are checkpointed in memory so a re-run never re-ingests. The development team always opens the day to actionable, scope-bounded items.

Workflow 1.2 — Daily Report

Process	Fires at 09:00 BDT every day. Reads the last 24 hours of activity from memory (recall_activity, recall_test_runs) and pulls the latest test artifacts from the four sibling MVP Access test projects.
Action	Claude composes an executive-style summary, a per-suite section for every test family that ran in the window, and a CICD section that highlights regressions against the seven-day trend. ReportLab renders the unified PDF with consistent typography and a NAPCO-branded header.
Result	A single email goes to the team via SMTP at ~09:05 BDT, carrying one PDF attachment that covers Requirement Management, API Functional, API Integration, API Load, E2E, and CICD in one place. The same digest can be optionally posted to a Teams channel.

Dimension 2 — Test Automation

Continuous quality on the MVP Access platform.

Workflow 2.1 — API Functional Test

Process	Triggered on demand via workflow_dispatch (typically before a release cut). Drives the Newman/Postman suite that lives in MVP-Access-API-Test.
Action	Runs the full functional contract suite, captures responses and assertions, and asks Claude to classify any failures (true regression, data-drift, environment).
Result	Writes a structured run record to test_run_history and stages a section for the next morning's Daily Report. No standalone email — the consolidated 09:00 BDT digest carries the result.

Workflow 2.2 — API Integration Test

Process	Fires at 02:00 BDT every day. Runs the pytest integration suite (MVP-Access-API-Test) end-to-end against the staging stack.
Action	Compares the run against a seven-day trend stored in memory; Claude flags newly failing endpoints versus flaky-but-known ones, and notes throughput shifts in the report.
Result	Stages an Integration section for the Daily Report and persists a trend-able row in test_run_history. Repeated failures across days promote the case to the regression callout in the digest.

Workflow 2.3 — API Load Test

Process	Fires at 03:00 BDT every Monday. Drives the multi-tier Locust suite (MVP-Access-API-Test) that ramps load to find the capacity ceiling.
Action	Captures per-tier latency, RPS, and failure rate; Claude annotates the result against the previous Monday's ceiling and notes any regression in the saturation point.
Result	Weekly capacity row written to test_run_history. Monday's Daily Report carries the load section so leadership sees the platform's headroom in one place each week.

Workflow 2.4 — MVP Access E2E Test

Process	Fires at 04:00 BDT every day. Drives Playwright suites across MVP-Access-E2E-Test (full), Easy-E2E-Test (smoke), and Release-Test (release candidates).
Action	Captures screenshots on failure and routes them as PDF inline images. Claude reads the trace and proposes the most likely root cause (selector drift, data setup, server 5xx) for each failed scenario.
Result	Stages an E2E section for the Daily Report with failure thumbnails embedded. The morning digest is one click away from the offending step.

Dimension 3 — DevOps / Release Automation

Fresh builds, deployed before the test cycle starts.

Workflow 3.1 — MVP Access Build & Deploy (CICD)

Process	Fires at 22:00 BDT every day on the self-hosted Windows runner. Pulls the latest MVP Access application code from on-prem TFS using the configured TFS_URL and a service account credential.
Action	Builds the .NET solution with MSBuild on the same runner, then deploys the build output to the IIS server over a UNC share at IIS_DEPLOY_PATH. A concurrency group prevents overlapping build-deploy runs.
Result	Emails a deploy-complete notification to the team. The 22:00 BDT slot is intentionally upstream of the 02:00 / 04:00 BDT test cycle, so the API Integration, Load, and E2E suites always exercise the freshest build of the day.

Technology Stack

Every component currently in production.

Category	Components
Reasoning	Claude Opus 4.7 via the local Claude CLI. Claude Agent SDK (claude-agent-sdk) for the agent loop. MCP tool servers for the 36 in-house tools.
Orchestration	GitHub Actions on a self-hosted Windows runner. Seven workflow files: six task workflows plus a probe-runner. Cron schedules for daily and weekly cadence; workflow_dispatch for manual triggers.
Runtime	Python 3.10+ via the py launcher. Windows Server VM at AEL\samin. Local Claude CLI authenticated against the engineer's Claude Max subscription.
Persistent memory	SQLite + FTS5 (nucleus_memory.db). Tables: activity_logs, requirements_seen, test_run_history, email_checkpoints, drive_processed. The DB is committed to the repo so memory survives runner replacement.
Inputs — Drive	google-api-python-client + google-auth (service account). Audio routed to Groq Whisper (whisper-large-v3-turbo, 25 MB cap). PDFs parsed with pypdf.
Inputs — mail	Gmail IMAP polling on a sender allowlist. Power Automate forwards Microsoft Teams channel messages into the same allowlist so one ingester covers both.
Test runners	Newman (Postman) for functional API. pytest for integration. Locust for load. Playwright for E2E across the three sibling test projects.
Build & deploy	MSBuild against the .NET solution on the self-hosted Windows runner. On-prem TFS for source pull. IIS deployment over a UNC share to the production-style staging server.
Reporting	ReportLab for PDF generation. SMTP over TLS (Gmail / Workspace) for delivery. Optional Microsoft Teams webhook for the morning digest.
Backlog sink	GitLab v4 REST API (thin requests wrapper). Three-hour atomic tasks pushed into the project backlog with title-based idempotent dedup.
Developer tooling	GitHub CLI (gh), Git, PowerShell + Bash inside workflow steps, RDP for VM access, .env contract documented in .env.example.

Prerequisites & Open Items

What we need from IT and leadership to keep this running.

DevOps / Release Automation (CI/CD)

- Dedicated deployment server (IIS host) with a stable UNC path for the build output.
- TFS service-account credentials (TFS_USERNAME, TFS_PASSWORD) with read access on \$/MVPAccess.
- Network openings: VM → on-prem TFS, and VM → IIS server over SMB.
- TFS client (tf.exe / Team Explorer Everywhere) installed on the self-hosted runner.
- Matching .NET Framework Developer Pack and MSBuild version on the runner.
- Antivirus exclusions on the build workspace and the deploy share (Defender / ESET routinely quarantines fresh DLLs).
- IIS app-pool ownership — confirmation of who recycles the pool after each deploy.
- Documented rollback procedure for a bad deploy (no automatic rollback today).
- Build artifact retention so older versions can be redeployed without rebuilding from TFS history.

Test Automation

- Stable staging environment URL and test-data accounts that are not silently rotated.
- Defined cadence or trigger for refreshing the staging database so tests do not drift into false greens.
- Sandbox or load-test target separate from staging so weekly Locust runs do not disturb manual QA.

Project Management

- Power Automate license and flow that forwards Teams channel posts into the IMAP allowlist.
- Google Workspace service account with Drive scope on a team-owned requirement folder (not a personal account).
- GitLab project token with *api* scope, owned by a team account so it survives an individual leaving.
- Allowlisted stakeholder / client email senders kept current as projects onboard.

Infrastructure (cross-cutting)

- VM uptime SLA — the AEL\samin VM must stay on 24/7 with autologon (no formal SLA today).
- Runner-service health monitoring beyond the hourly probe-runner (alert when the GitHub Actions service stops on the VM).
- Documented disaster-recovery procedure if the VM is destroyed (today: runner re-registration and secret reapply are manual).
- Credential rotation policy for TFS, GitLab, Google service account, SMTP, and Groq.

Process / Governance

- On-call ownership for “the morning email did not arrive” or “the deploy notification did not come through.”
- Incident escalation path if the agent silently produces bad output for two days running.
- Access-review cadence to confirm service accounts only retain the access they actually need.

Operational Status

All seven task workflows are live in production today.

Workflow	Schedule	Status	Notes
1.1 Requirement Management	Every 2h, 09–17 BDT, Sun–Thu	LIVE	IMAP + Drive ingest, Whisper transcription, FTS5 dedup, GitLab issue push with title-keyed idempotency.
1.2 Daily Report	09:00 BDT daily	LIVE	Single consolidated PDF email covering all six dimensions; optional Teams digest via webhook.
2.1 API Functional Test	workflow_dispatch	LIVE	Newman/Postman suite, Claude failure-classification, run recorded to test_run_history.
2.2 API Integration Test	02:00 BDT daily	LIVE	pytest integration suite vs 7-day trend; regressions promoted to morning digest.
2.3 API Load Test	03:00 BDT Mondays	LIVE	Multi-tier Locust suite; capacity ceiling tracked week over week.
2.4 MVP Access E2E Test	04:00 BDT daily	LIVE	Playwright Full + Easy + Release variants; failure screenshots embedded in the daily PDF.
3.1 MVP Access Build & Deploy	22:00 BDT daily	LIVE	TFS pull → MSBuild → IIS deploy over UNC. Upstream of the test cycle so 02:00 / 04:00 BDT suites exercise the freshest build.
— Probe Runner	Hourly	LIVE	Health check on the self-hosted runner; alerts if the VM falls off the actions queue.

One repo, one VM, one morning email. The platform is built to grow: additional dimensions can be added by writing a prompt and registering tools, without disturbing the workflows already running. Next on the roadmap: a release-readiness dimension that joins the API and E2E signals into a single go / no-go recommendation per release candidate.